Research on robot arena Exploration and mapping based on Bug1 algorithm

Introduction

This study aims to achieve autonomous exploration and occupancy grid map construction of unknown arenas (PA1_simple.wbt and PA1_complex.wbt) by using the Pioneer 3-DX robot in the Webots simulation environment. The core task is to design a general navigation strategy and collision avoidance mechanism to ensure that the robot can complete exploration without preloading map information. In the research process, the anti-collision monitoring logic is iterated, and the stable navigation is realized by combining Bug1 algorithm and PID control, and the high-precision occupancy grid map is finally generated.

1 Exploration and Navigation Strategy Design

1.1. Explore strategies

The exploration strategy adopts the logic of "waypoint planning + coverage loop closure". Waypoint planning is to design two types of waypoints covering the arena including outer ring waypoints (-4.5,4.5 → 4.5,3.5 → 4.5,-4.5 → -4.5,-4.5 → -4.0,-4.0), ensuring boundary exploration. Oblique waypoints (-4.0,-4.0→4.2,3), complementing internal area coverage. The waypoints are switched by set_goal(), and the arrival determination tolerance is set to 0.1m. OccupancyGrid class is used to calculate the proportion of explored cells in real time, when the coverage rate is lower than 90%, the waypoint sequence is automatically reset to re-explore to avoid local omission.

But in fact, the coverage has reached 96% after the end of waypoint, so the coverage loop closure is not applied.

1.2. Navigation core -Bug1 algorithm and PID wall following

Based on the Bug1 algorithm framework of Lab6, the robot state switching and obstacle avoidance are realized. There are two main states of the robot including GOALSEEKING and WALLFOLLOWING. During target tracking, the wheel speed was adjusted according to the target bearing through adjust_velocity(), and the acceleration (from 3 to the maximum speed within 0.4m from the starting point) and deceleration (from the maximum to 1 within 0.8m from the target) logic were combined. When the obstacle in front (distance $\leqslant$0.3m) is detected, it switches to the wall

following state.

A PID controller (kp=6.0, kd=40.0, ki=0.0) is used in the wall following phase to maintain a 0.3m wall sticking distance through the left-facing sensor (s0). When facing obstacles in front (distance ⩽0.27m), turn right to avoid obstacles; when no wall is detected, turn left to find the wall to ensure stable movement along the boundary of obstacles, and solve the problem of wall following oscillation in Lab6.

2. Challenges and Reflections

2.1 Frequent "stuck wall" in robotic exploration

For the frequent "stuck wall" problem in robot exploration (e.g., stuck at the obstacle Angle in WF_LEAVE state, stuck along the wall in WF_SEARCH state), it takes several solution in 3 iteration including Reversing, Reversing + steering, and Reverse steering + temporary wall following.

Specifically, Reversing is that stuck wall was determined by detecting "no change in the distance to the target/impact point (<0.001m)" for 50 consecutive frames, and 30 frames of reversing were triggered. This scheme can solve the plane stuck problem, but it is not effective for corner obstacles.

Reversing + steering means for the WF_LEAVE state (with the highest probability of stuck wall), after detecting no change in the distance of 20 frames, it was optimized to retreat from obstacle contact for 90% of the time for starting 70 frames of self-rescue. Turn right in the last 10% of the time and deviate from the current obstacle direction. This solution resolves the issue of getting stuck at the corner wall, but it is still possible to get stuck again due to "finding a leave point".

Reverse steering + temporary wall following means on the basis of the second generation, "300 frames temporarily move along the wall" is added. After the self-rescue, the robot enters the temp_wallfollow_in_leave mode, maintains the left side of the wall (0.3m) through the PID controller, turns right when the obstacle in front, turns left when there is no wall to find the wall. This upgrade ensures re-connect to stable navigation after self-rescue, essentially avoiding the problem of getting stuck completely.

2.2 Poor map stability

In the original inverse sensor model, when the sensor returns the maximum range (mr == max_range), the corresponding grid is directly marked as "free" (logodds = self.lfree). This causes the grid to be marked as "occupied" when the robot is near an obstacle. However, when the robot

moved away, the sensor returned to the maximum distance due to the range limit, and the original wall grid was updated to be "free".

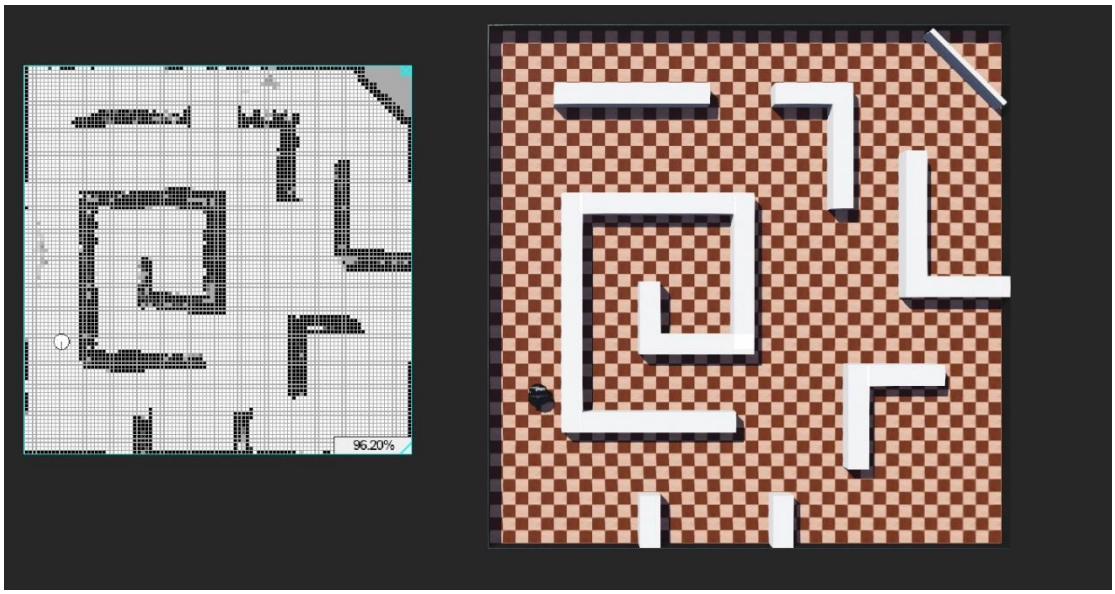
To solve the above problem, when the sensor returns the maximum distance, the grid is no longer marked as "free", but its prior probability (logodds = self.lprior) is maintained to avoid miscovering the marked wall grid when it is far from the obstacle.

A weight (edge_factor) is applied to the "free update" of edge of sensor cone angle, and the update intensity of the center ray is 100%, and the edge ray retains a minimum of 20% intensity. This method not only ensures the recognition accuracy of the free area in the center direction of the sensor, but also avoids the blurring of the wall contour caused by the wrong update of edge ray.

Only "near the sensor hit point" is marked as occupation, "in front of the hit point" is marked as weight reduction freedom, and "behind the hit point" and "maximum range area" are kept prior, which strictly limits the scope of raster update and ensures the stability of the map.

3. Experimental results

The results shows that the coverage rate is 96.20%, which satisfies the termination condition. In complex scenes, the robot successfully bypassed multiple scattered obstacles with complete contours of obstacle.



Conclusion

In this study, through the exploration framework of "waypoint + coverage" and the anti-collision mechanism, combined with Bug1 and PID control, the robot realizes the universal exploration of

unknown arena. The final map coverage is over 90%.